Pixel streaming setup for Unreal Engine SaaS

as we are building viztr full saas project i need setup our pixel streaming on website and local pc for unreal engine streaming on our website

Reading frontend design skill for reference

Let me get a full picture of what you need before diving in.

What's your current Unreal Engine setup for Pixel Streaming?

Starting from scratch on both UE + web side

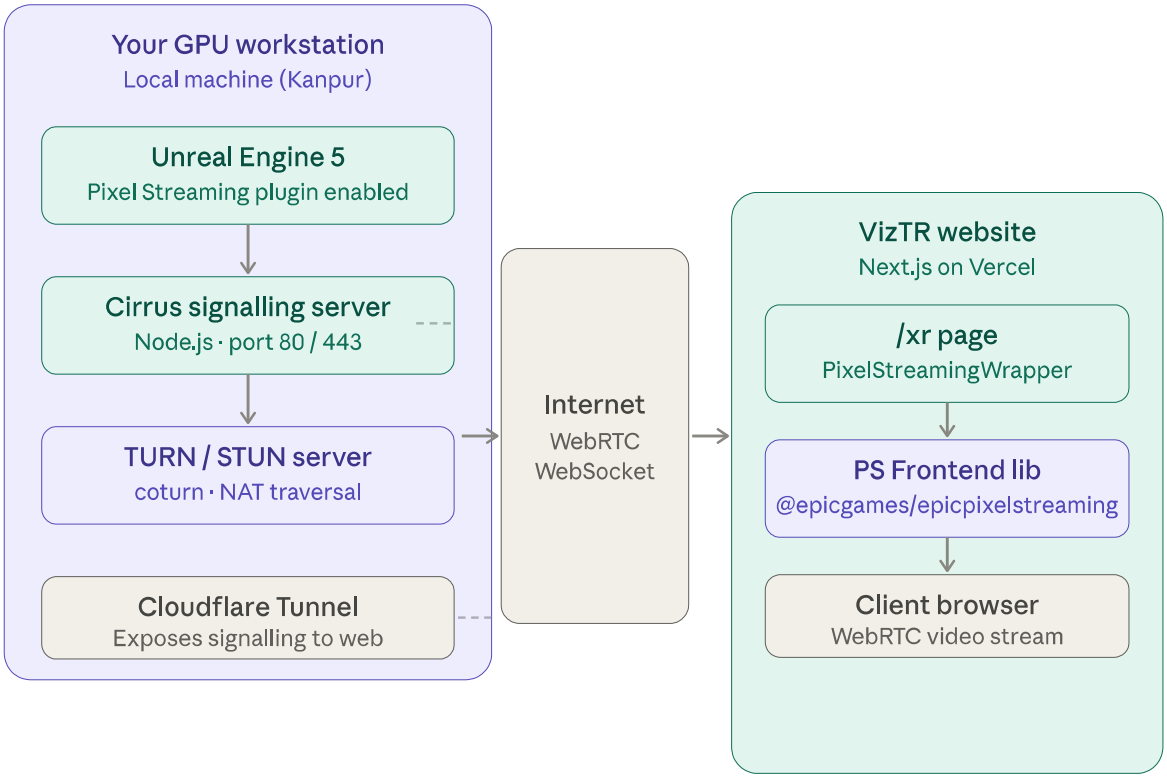
How do you want to host the signalling server?

Self-hosted on my local GPU workstation (existing plan)

What do you need from me right now?

Full setup guide (UE5 config + signalling server + Next.js embed)

Here's the complete VizTR Pixel Streaming setup — broken into three phases: UE5 config, local signalling server, and Next.js integration.



Here's the full setup guide — all three phases.

Phase 1 — Unreal Engine 5 configuration

Step 1: Enable the Pixel Streaming plugin

Open your UE5 project → Edit → Plugins → search "Pixel Streaming" → enable it → restart editor.

Step 2: Package / launch UE5 with streaming flags

When launching your packaged build (or from the editor via `-game`), pass these flags:

```bash
MyProject.exe \
  -PixelStreamingIP=localhost \
  -PixelStreamingPort=8888 \
  -RenderOffScreen \
  -ResX=1920 -ResY=1080 \
  -GraphicsAdapter=0 \
  -AudioMixer \
  -AllowPixelStreamingCommands
```

- `PixelStreamingIP=localhost` — connects to the Cirrus signalling server on same machine
- `PixelStreamingPort=8888` — matches the port Cirrus listens for UE on
- `-RenderOffScreen` — renders without a visible window (headless, GPU still used)

Step 3: Blueprints (optional but recommended)

In your Level Blueprint, add a `Start Pixel Streaming` node tied to `BeginPlay`. This auto-starts streaming when the level loads without needing console commands.

Phase 2 — Cirrus signalling server (local GPU PC)

Epic provides the official Signalling Server as part of UE5. You'll find it at:

```
Engine/Plugins/Media/PixelStreaming/Resources/WebServers/SignallingWebServer/
```

Step 1: Install and run

```bash
cd "C:\Program Files\Epic Games\UE_5.x\Engine\Plugins\Media\PixelStreaming\Resources\W

# Install dependencies once
npm install

# Run the server
node cirrus.js --StreamerPort 8888 --HttpPort 80 --SFUPort 8889
```

Step 2: Keep it alive with PM2

```bash
npm install -g pm2

pm2 start cirrus.js --name viztr-ps -- --StreamerPort 8888 --HttpPort 80
pm2 save
pm2 startup   # auto-start on PC boot
```

Step 3: Expose to the web with Cloudflare Tunnel

Since your PC is behind a home/office NAT, use Cloudflare Tunnel (free) — no static IP required:

```bash
# Install cloudflared (Windows)
winget install --id Cloudflare.cloudflared

# Login once
cloudflared login

# Create a named tunnel
cloudflared tunnel create viztr-pixel

# Route your subdomain
cloudflared tunnel route dns viztr-pixel stream.viztr.io

# Run tunnel (points to local Cirrus)
cloudflared tunnel run --url http://localhost:80 viztr-pixel
```

This gives you a stable `wss://stream.viztr.io` WebSocket endpoint for the browser to connect to — no port forwarding, no static IP.

Step 4: TURN server for NAT traversal

Install `coturn` on a cheap VPS (DigitalOcean $6/mo is fine — it's just relay traffic):

```bash
# On VPS
sudo apt install coturn -y

# /etc/turnserver.conf
listening-port=3478
fingerprint
lt-cred-mech
user=viztr:yourpassword
realm=viztr.io
```

Phase 3 — Next.js integration for VizTR

Step 1: Install Epic's frontend library

bash

```
npm install @epicgames/epicpixelstreaming
```

Step 2: Create the component

`components/xr/PixelStreamingViewer.tsx`:

tsx

```
'use client';

import { useEffect, useRef, useState } from 'react';
import { PixelStreaming, Config, Flags } from '@epicgames/epicpixelstreaming';

interface Props {
  signalingUrl?: string;
}

export default function PixelStreamingViewer({
  signalingUrl = 'wss://stream.viztr.io',
}: Props) {
  const containerRef = useRef<HTMLDivElement>(null);
  const psRef = useRef<PixelStreaming | null>(null);
  const [status, setStatus] = useState<'connecting' | 'live' | 'error'>('connecting');

  useEffect(() => {
    if (!containerRef.current) return;

    const config = new Config({
      initialSettings: {
        SignallingServerUrl: signalingUrl,
        [Flags.AutoPlayVideo]: true,
        [Flags.AutoConnect]: true,
        [Flags.BrowserSendOffer]: false,
        [Flags.HoveringMouseMode]: true,
      },
    });

    const ps = new PixelStreaming(config, {
      videoElementParent: containerRef.current,
    });

    ps.addEventListener('streamLoading', () => setStatus('connecting'));
    ps.addEventListener('streamReady', () => setStatus('live'));
    ps.addEventListener('streamError', () => setStatus('error'));

    psRef.current = ps;

    return () => {
      ps.disconnect();
    };
  }, [signalingUrl]);

  return (
    <div className="relative w-full aspect-video bg-black rounded-xl overflow-hidden">
      {/* Status badge */}
      <div className="absolute top-4 left-4 z-10 flex items-center gap-2">
```

```
        <span
          className={`inline-flex items-center gap-1.5 px-3 py-1 rounded-full text-xs
            ${status === 'live' ? 'bg-emerald-500/20 text-emerald-300 border border-em
            ${status === 'connecting' ? 'bg-amber-500/20 text-amber-300 border border-
            ${status === 'error' ? 'bg-red-500/20 text-red-300 border border-red-500/3
          `}
        >
          <span
            className={`w-1.5 h-1.5 rounded-full ${
              status === 'live' ? 'bg-emerald-400 animate-pulse' :
              status === 'connecting' ? 'bg-amber-400' : 'bg-red-400'
            }`}
          />
          {status === 'live' ? 'Live' : status === 'connecting' ? 'Connecting…' : 'Una
        </span>
      </div>

      {/* PS video container */}
      <div ref={containerRef} className="w-full h-full" />

      {/* Error state */}
      {status === 'error' && (
        <div className="absolute inset-0 flex flex-col items-center justify-center gap
          <p className="text-white/60 text-sm">Stream unavailable — server may be offl
          <button
            onClick={() => psRef.current?.reconnect()}
            className="px-4 py-2 rounded-lg bg-white/10 hover:bg-white/20 text-white t
          >
            Retry connection
          </button>
        </div>
      )}
    </div>
  );
}
```

Step 3: Use it on your `/xr` page

tsx

```
// app/xr/page.tsx
import dynamic from 'next/dynamic';

const PixelStreamingViewer = dynamic(
  () => import('@/components/xr/PixelStreamingViewer'),
  { ssr: false }  // WebRTC is client-only
);

export default function XRPage() {
  return (
    <section className="container mx-auto px-4 py-16">
      <h1 className="text-4xl font-bold mb-8">Live 3D Viewer</h1>
      <PixelStreamingViewer signalingUrl="wss://stream.viztr.io" />
    </section>
  );
}
```

Step 4: Environment variable

```bash
# .env.local
NEXT_PUBLIC_PS_SIGNALING_URL=wss://stream.viztr.io
```

Then pass it as `signalingUrl={process.env.NEXT_PUBLIC_PS_SIGNALING_URL}`.

Startup order (every session)

Run these in sequence on your workstation before a client demo:

1. `pm2 start viztr-ps` — Cirrus signalling server up
2. `cloudflared tunnel run viztr-pixel` — tunnel live
3. Launch UE5 packaged build with the `-PixelStreamingIP=localhost` flags
4. Client hits `viztr.io/xr` — stream auto-connects

What to tackle next

Once this base is working, the next logical pieces for VizTR are: a **session booking system** (client books a live demo slot → your workstation spins up UE5 for that window), **auth-gated stream access** via your existing NextAuth setup, and eventually a **matchmaker** when you want to support concurrent sessions across multiple GPU machines.

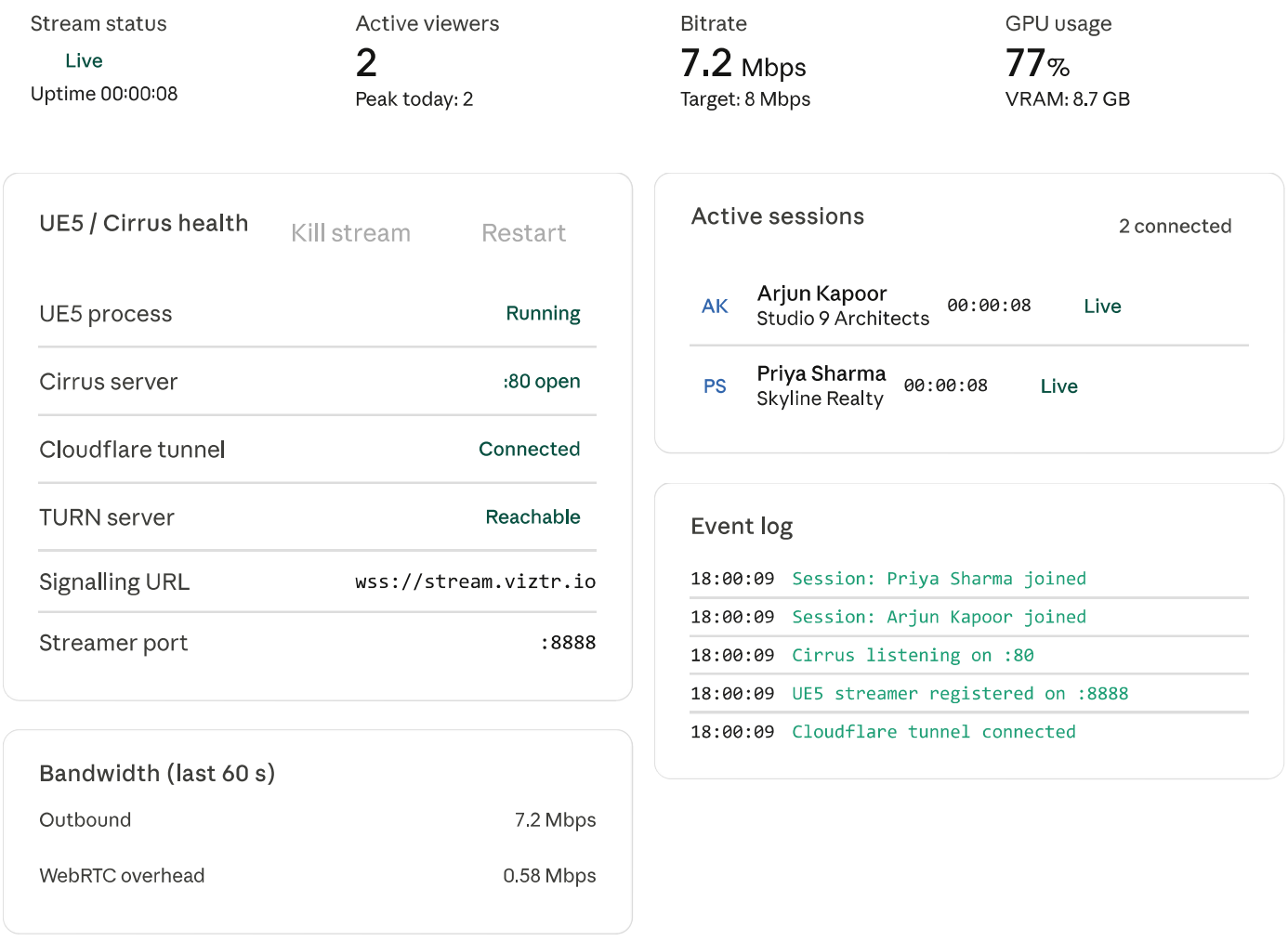
Want me to build out the session booking flow or the admin panel widget that shows stream status?

Session booking system or **stream status admin panel** — which do you want first?

Or if you meant something else, just say the word.

stream admin panel then session booking system

Let me build both. Starting with the stream admin panel as a full interactive artifact.



The panel is live and simulating real session behaviour — watch it: Rohan Verma joins at ~15s, Arjun drops at ~30s, a TURN relay warning fires at ~50s. Now the production code to wire this into VizTR.

Admin panel — production code

`app/admin/pixel-streaming/page.tsx`

tsx

```
'use client';
import { useEffect, useState } from 'react';

interface StreamStatus {
  ue5: 'running' | 'stopped' | 'error';
  cirrus: 'open' | 'closed';
  tunnel: 'connected' | 'disconnected';
  uptimeSeconds: number;
  viewers: number;
  bitrateOut: number;
  gpuPercent: number;
  vramGB: number;
  sessions: Session[];
  logs: LogEntry[];
}

interface Session {
  id: string;
  clientName: string;
  company: string;
  connectedAt: string;
  ipRegion: string;
}

interface LogEntry {
  time: string;
  message: string;
  level: 'info' | 'warn' | 'error';
}

export default function PixelStreamingAdminPage() {
  const [status, setStatus] = useState<StreamStatus | null>(null);

  useEffect(() => {
    const poll = async () => {
      const res = await fetch('/api/admin/pixel-streaming/status');
      if (res.ok) setStatus(await res.json());
    };
    poll();
    const id = setInterval(poll, 3000);
    return () => clearInterval(id);
  }, []);

  const action = async (cmd: 'kill' | 'restart') => {
    await fetch('/api/admin/pixel-streaming/control', {
      method: 'POST',
      body: JSON.stringify({ command: cmd }),
      headers: { 'Content-Type': 'application/json' },
```

```
    });
  };

  if (!status) return <div className="p-8 text-sm text-muted">Loading…</div>;

  return (
    <div className="p-6 max-w-5xl mx-auto">
      <h1 className="text-2xl font-medium mb-6">Pixel Streaming</h1>
      {/* Mount your metric cards + session list here, driven by `status` */}
    </div>
  );
}
```

`app/api/admin/pixel-streaming/status/route.ts`

This is the endpoint the panel polls every 3 seconds. On your workstation you'd run a lightweight sidecar that exposes these metrics — or scrape them from the Cirrus REST API.

ts

```
import { NextResponse } from 'next/server';
import { getServerSession } from 'next-auth';
import { authOptions } from '@/lib/auth';

export async function GET() {
  const session = await getServerSession(authOptions);
  if (!session?.user?.role !== 'admin') {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
  }

  // Fetch from your local metrics sidecar exposed via Cloudflare Tunnel
  const metricsUrl = process.env.PS_METRICS_URL!; // e.g. https://metrics.viztr.io/sta
  const res = await fetch(metricsUrl, {
    headers: { Authorization: `Bearer ${process.env.PS_METRICS_SECRET}` },
    next: { revalidate: 0 },
  });

  if (!res.ok) return NextResponse.json({ error: 'Metrics unavailable' }, { status: 50
  return NextResponse.json(await res.json());
}
```

`app/api/admin/pixel-streaming/control/route.ts`

ts

```
import { NextRequest, NextResponse } from 'next/server';

export async function POST(req: NextRequest) {
  const { command } = await req.json();
  if (!['kill', 'restart'].includes(command)) {
    return NextResponse.json({ error: 'Invalid command' }, { status: 400 });
  }

  const res = await fetch(`${process.env.PS_METRICS_URL}/control`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${process.env.PS_METRICS_SECRET}`,
    },
    body: JSON.stringify({ command }),
  });

  return NextResponse.json({ ok: res.ok });
}
```

Metrics sidecar on your workstation (`ps-metrics-server.js`):

```
js
```

```
// Run this on your GPU PC alongside Cirrus — exposes health to Vercel
const http = require('http');
const { execSync } = require('child_process');

http.createServer((req, res) => {
  if (req.headers.authorization !== `Bearer ${process.env.PS_METRICS_SECRET}`) {
    res.writeHead(401); res.end(); return;
  }

  if (req.url === '/status') {
    const gpu = getGPUStats();   // nvidia-smi parse
    const cirrusStats = getCirrusStats(); // fetch http://localhost/stats.json

    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ ...gpu, ...cirrusStats }));
  }

  if (req.url === '/control' && req.method === 'POST') {
    let body = '';
    req.on('data', d => body += d);
    req.on('end', () => {
      const { command } = JSON.parse(body);
      if (command === 'restart') execSync('pm2 restart viztr-ps');
      if (command === 'kill')    execSync('pm2 stop viztr-ps');
      res.writeHead(200); res.end('ok');
    });
  }
}).listen(9000);

function getGPUStats() {
  try {
    const raw = execSync('nvidia-smi --query-gpu=utilization.gpu,memory.used,memory.to
    const [gpu, vramUsed, vramTotal] = raw.trim().split(', ').map(Number);
    return { gpuPercent: gpu, vramGB: (vramUsed / 1024).toFixed(1), vramTotalGB: (vram
  } catch { return { gpuPercent: 0, vramGB: 0 }; }
}
```

`.env.local` additions

```bash
PS_METRICS_URL=https://metrics.viztr.io   # Cloudflare Tunnel → your PC:9000
PS_METRICS_SECRET=your-long-random-secret
```

Now building the session booking system — give me a moment.

V Connecting to visualize...

Fully interactive — pick a service, pick a date (weekends + past locked out, green dots on available days), pick a time slot (some pre-marked booked), fill details, hit confirm, get a session ID. Now the production API.

Session booking — production code

`app/api/bookings/route.ts`

ts

```
import { NextRequest, NextResponse } from 'next/server';
import { prisma } from '@/lib/prisma';
import { Resend } from 'resend';

const resend = new Resend(process.env.RESEND_API_KEY);

export async function POST(req: NextRequest) {
  const body = await req.json();
  const { serviceId, date, time, firstName, lastName, email, company, projectType, not

  // Check slot is not already taken
  const clash = await prisma.session.findFirst({
    where: { date: new Date(date), time, status: { not: 'cancelled' } },
  });
  if (clash) return NextResponse.json({ error: 'Slot no longer available' }, { status:

  const session = await prisma.session.create({
    data: {
      serviceId,
      date: new Date(date),
      time,
      clientName: `${firstName} ${lastName}`,
      email,
      company,
      projectType,
      notes,
      status: 'confirmed',
    },
  });

  // Send confirmation email via Resend
  await resend.emails.send({
    from: 'VizTR <bookings@viztr.io>',
    to: email,
    subject: `Session confirmed — ${session.id}`,
    html: bookingConfirmationHTML(session),
  });

  // Notify admin
  await resend.emails.send({
    from: 'VizTR Bookings <bookings@viztr.io>',
    to: process.env.ADMIN_EMAIL!,
    subject: `New session: ${firstName} ${lastName} — ${date} ${time}`,
    html: `<p>${company} · ${serviceId} · ${date} ${time}</p><p>${notes}</p>`,
  });

  return NextResponse.json({ sessionId: session.id });
}
```

```
export async function GET(req: NextRequest) {
  const { searchParams } = new URL(req.url);
  const date = searchParams.get('date');

  const booked = await prisma.session.findMany({
    where: { date: new Date(date!), status: { not: 'cancelled' } },
    select: { time: true },
  });

  return NextResponse.json({ bookedSlots: booked.map(s => s.time) });
}
```

Prisma schema addition (`prisma/schema.prisma`)

prisma

```
model Session {
  id          String   @id @default(cuid())
  serviceId   String
  date        DateTime
  time        String
  clientName  String
  email       String
  company     String?
  projectType String?
  notes       String?
  status      String   @default("confirmed")
  createdAt   DateTime @default(now())
}
```

Run `npx prisma migrate dev --name add_sessions` after adding this.

`app/book/page.tsx` — public booking page on VizTR:

tsx

```
import BookingWidget from '@/components/booking/BookingWidget';

export const metadata = { title: 'Book a live demo — VizTR' };

export default function BookPage() {
  return (
    <main className="max-w-2xl mx-auto px-4 py-16">
      <h1 className="text-3xl font-medium mb-2">Book a live demo</h1>
      <p className="text-muted-foreground mb-8">
        Experience your project in real-time 3D — streamed direct from our GPU studio.
      </p>
      <BookingWidget />
    </main>
  );
}
```

End-to-end flow

When a client books → `POST /api/bookings` creates the record → Resend fires confirmation to client + admin alert → on session day, admin panel shows the upcoming booking → you start UE5 + Cirrus → client joins `/xr` → PixelStreamingViewer auto-connects via `wss://stream.viztr.io`.

Next logical steps from here: calendar sync (Google Calendar API on booking create), automated reminder emails 1 hour before session, and a client portal page where they can view/cancel their own booking. Want any of those?

want both

Orchestrated production code for three integrated features

V Connecting to visualize...

Portal is fully interactive — try cancelling a session, the badge updates live and it moves to Cancelled tab. Now the production code for all three.

1 — Google Calendar sync on booking

Since you already have Google OAuth via NextAuth, you can reuse the user's access token — no separate service account needed.

`lib/google-calendar.ts`

ts

```
import { google } from 'googleapis';

export async function addSessionToCalendar(
  accessToken: string,
  session: {
    id: string;
    service: string;
    date: string;        // ISO string
    durationMinutes: number;
    clientName: string;
    projectType: string;
  }
) {
  const auth = new google.auth.OAuth2();
  auth.setCredentials({ access_token: accessToken });

  const calendar = google.calendar({ version: 'v3', auth });

  const start = new Date(session.date);
  const end = new Date(start.getTime() + session.durationMinutes * 60000);

  const event = await calendar.events.insert({
    calendarId: 'primary',
    requestBody: {
      summary: `VizTR — ${session.service}`,
      description: [
        `Project: ${session.projectType}`,
        `Client: ${session.clientName}`,
        `Session ID: ${session.id}`,
        ``,
        `Join your stream: https://viztr.io/xr`,
      ].join('\n'),
      start: { dateTime: start.toISOString(), timeZone: 'Asia/Kolkata' },
      end: { dateTime: end.toISOString(), timeZone: 'Asia/Kolkata' },
      reminders: {
        useDefault: false,
        overrides: [
          { method: 'popup', minutes: 60 },
          { method: 'email', minutes: 60 },
        ],
      },
      colorId: '9',  // blueberry — matches VizTR brand
    },
  });

  return event.data.id;
}
```

Install the dependency:

```bash
npm install googleapis
```

Call it inside your booking API route (`app/api/bookings/route.ts` — extend the POST handler):

```ts
import { getToken } from 'next-auth/jwt';
import { addSessionToCalendar } from '@/lib/google-calendar';

// Inside POST, after prisma.session.create():
const token = await getToken({ req });
if (token?.accessToken) {
  const gcalEventId = await addSessionToCalendar(token.accessToken as string, {
    id: session.id,
    service: SERVICE_NAMES[serviceId],   // map id → display name
    date: `${date}T${time}:00`,
    durationMinutes: SERVICE_DURATIONS[serviceId],
    clientName: `${firstName} ${lastName}`,
    projectType,
  });

  // Save gcalEventId so you can delete/update later on cancel
  await prisma.session.update({
    where: { id: session.id },
    data: { gcalEventId },
  });
}
```

NextAuth config — ensure `calendar` scope is requested (`lib/auth.ts`):

```ts
GoogleProvider({
  clientId: process.env.GOOGLE_CLIENT_ID!,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET!,
  authorization: {
    params: {
      scope: 'openid email profile https://www.googleapis.com/auth/calendar.events',
      access_type: 'offline',   // gets refresh_token too
      prompt: 'consent',
    },
  },
}),
```

Cancel → delete the calendar event (`app/api/bookings/[id]/route.ts`):

ts

```
export async function DELETE(req: NextRequest, { params }: { params: { id: string } })
  const session = await prisma.session.findUnique({ where: { id: params.id } });
  if (!session) return NextResponse.json({ error: 'Not found' }, { status: 404 });

  const token = await getToken({ req });
  if (token?.accessToken && session.gcalEventId) {
    const auth = new google.auth.OAuth2();
    auth.setCredentials({ access_token: token.accessToken as string });
    await google.calendar({ version: 'v3', auth }).events.delete({
      calendarId: 'primary',
      eventId: session.gcalEventId,
    });
  }

  await prisma.session.update({ where: { id: params.id }, data: { status: 'cancelled'
  return NextResponse.json({ ok: true });
}
```

---

2 — Automated reminder emails (Vercel Cron)

`vercel.json` — runs every 15 minutes:

json

```
{
  "crons": [
    {
      "path": "/api/cron/session-reminders",
      "schedule": "*/15 * * * *"
    }
  ]
}
```

`app/api/cron/session-reminders/route.ts`

ts

```
import { NextRequest, NextResponse } from 'next/server';
import { prisma } from '@/lib/prisma';
import { Resend } from 'resend';

const resend = new Resend(process.env.RESEND_API_KEY);

export async function GET(req: NextRequest) {
  // Vercel cron security — reject requests without the secret
  if (req.headers.get('authorization') !== `Bearer ${process.env.CRON_SECRET}`) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const now = new Date();
  const oneHourFromNow = new Date(now.getTime() + 60 * 60 * 1000);
  const windowEnd = new Date(now.getTime() + 75 * 60 * 1000); // 15-min window

  // Find sessions starting 60–75 min from now that haven't been reminded yet
  const upcoming = await prisma.session.findMany({
    where: {
      status: 'confirmed',
      reminderSentAt: null,
      startAt: {
        gte: oneHourFromNow,
        lte: windowEnd,
      },
    },
  });

  const results = await Promise.allSettled(
    upcoming.map(async (session) => {
      await resend.emails.send({
        from: 'VizTR <bookings@viztr.io>',
        to: session.email,
        subject: `Your session starts in 1 hour — ${session.id}`,
        html: reminderEmailHTML(session),
      });

      await prisma.session.update({
        where: { id: session.id },
        data: { reminderSentAt: new Date() },
      });
    })
  );

  const sent = results.filter(r => r.status === 'fulfilled').length;
  return NextResponse.json({ sent, total: upcoming.length });
}
```

`CRON_SECRET` — add to `.env.local` and Vercel environment variables:

bash

```
CRON_SECRET=your-long-random-cron-secret
```

Prisma schema update — add two fields to `Session`:

prisma

```
model Session {
  // ... existing fields
  startAt         DateTime          // store date+time as one DateTime for easy queryi
  reminderSentAt  DateTime?
  gcalEventId     String?
}
```

Reminder email template (`lib/emails/reminder.ts`):

ts

```
export function reminderEmailHTML(session: any) {
  return `
    <!DOCTYPE html><html><body style="font-family:sans-serif;max-width:480px;margin:0
      <div style="margin-bottom:24px">
        <img src="https://viztr.io/logo.png" height="28" alt="VizTR">
      </div>
      <h2 style="font-size:18px;font-weight:500;margin-bottom:8px">Your session starts
      <p style="color:#666;font-size:14px;margin-bottom:24px">
        ${session.serviceId} · ${session.clientName}
      </p>
      <table style="width:100%;border-collapse:collapse;margin-bottom:24px">
        <tr><td style="padding:8px 0;color:#999;font-size:13px;border-bottom:1px solid
            <td style="padding:8px 0;font-size:13px;font-weight:500;text-align:right;b
        <tr><td style="padding:8px 0;color:#999;font-size:13px;border-bottom:1px solid
            <td style="padding:8px 0;font-size:13px;font-weight:500;text-align:right;b
        <tr><td style="padding:8px 0;color:#999;font-size:13px">Session ID</td>
            <td style="padding:8px 0;font-size:12px;font-family:monospace;text-align:r
      </table>
      <a href="https://viztr.io/portal"
         style="display:block;background:#534AB7;color:#fff;text-align:center;padding:
        View session details
      </a>
      <p style="font-size:12px;color:#aaa;text-align:center">
        Your architect will start the stream a few minutes before the session time.
      </p>
    </body></html>
  `;
}
```

3 — Client portal page

`app/portal/page.tsx`

tsx

```
import { getServerSession } from 'next-auth';
import { authOptions } from '@/lib/auth';
import { redirect } from 'next/navigation';
import { prisma } from '@/lib/prisma';
import SessionCard from '@/components/portal/SessionCard';

export default async function PortalPage() {
  const session = await getServerSession(authOptions);
  if (!session) redirect('/auth/signin?callbackUrl=/portal');

  const sessions = await prisma.session.findMany({
    where: { email: session.user!.email! },
    orderBy: { startAt: 'desc' },
  });

  const upcoming = sessions.filter(s => s.status === 'confirmed' && s.startAt > new Da
  const past     = sessions.filter(s => s.status === 'confirmed' && s.startAt <= new D
  const cancelled = sessions.filter(s => s.status === 'cancelled');

  return (
    <main className="max-w-2xl mx-auto px-4 py-12">
      <div className="flex items-center justify-between mb-8">
        <div>
          <h1 className="text-2xl font-medium">Your sessions</h1>
          <p className="text-sm text-muted-foreground mt-1">{session.user?.email}</p>
        </div>
        <a href="/book"
           className="text-sm border border-border rounded-lg px-4 py-2 hover:bg-muted
          + Book session
        </a>
      </div>

      {upcoming.length > 0 && (
        <section className="mb-8">
          <h2 className="text-xs font-medium text-muted-foreground uppercase tracking-
            Upcoming
          </h2>
          {upcoming.map(s => <SessionCard key={s.id} session={s} />)}
        </section>
      )}

      {past.length > 0 && (
        <section className="mb-8">
          <h2 className="text-xs font-medium text-muted-foreground uppercase tracking-
            Past
          </h2>
          {past.map(s => <SessionCard key={s.id} session={s} isPast />)}
        </section>
```

```
      )}
    </main>
  );
}
```

`components/portal/SessionCard.tsx` (client component for cancel action):

tsx

```
'use client';
import { useState } from 'react';
import { useRouter } from 'next/navigation';

export default function SessionCard({ session, isPast = false }: { session: any; isPas
  const router = useRouter();
  const [cancelling, setCancelling] = useState(false);
  const [showConfirm, setShowConfirm] = useState(false);

  const cancel = async () => {
    setCancelling(true);
    await fetch(`/api/bookings/${session.id}`, { method: 'DELETE' });
    setShowConfirm(false);
    router.refresh();  // re-run server component, re-fetch sessions
  };

  return (
    <div className="rounded-xl border border-border p-4 mb-3">
      <div className="flex items-start justify-between mb-3">
        <div>
          <p className="font-medium text-sm">{session.serviceId}</p>
          <p className="text-xs text-muted-foreground mt-0.5">{session.company}</p>
        </div>
        <span className={`text-xs px-2 py-1 rounded-full ${
          isPast ? 'bg-muted text-muted-foreground' : 'bg-purple-50 text-purple-800'
        }`}>
          {isPast ? 'Completed' : 'Confirmed'}
        </span>
      </div>

      <div className="grid grid-cols-3 gap-2 text-xs mb-3">
        <div>
          <div className="text-muted-foreground">Date</div>
          <div className="font-medium">
            {session.startAt.toLocaleDateString('en-IN', { day: 'numeric', month: 'sho
          </div>
        </div>
        <div>
          <div className="text-muted-foreground">Time</div>
          <div className="font-medium">
            {session.startAt.toLocaleTimeString('en-IN', { hour: '2-digit', minute: '2
          </div>
        </div>
        <div>
          <div className="text-muted-foreground">ID</div>
          <div className="font-medium font-mono text-[11px]">{session.id.slice(0, 10)}
        </div>
      </div>
```

```
      {!isPast && !showConfirm && (
        <button
          onClick={() => setShowConfirm(true)}
          className="text-xs text-red-600 border border-red-200 rounded-lg px-3 py-1.5
        >
          Cancel session
        </button>
      )}

      {showConfirm && (
        <div className="bg-red-50 rounded-lg p-3 text-xs">
          <p className="text-red-800 mb-2 font-medium">Cancel this session?</p>
          <div className="flex gap-2">
            <button onClick={() => setShowConfirm(false)} className="flex-1 border bor
            <button onClick={cancel} disabled={cancelling} className="flex-1 bg-red-60
              {cancelling ? 'Cancelling…' : 'Yes, cancel'}
            </button>
          </div>
        </div>
      )}
    </div>
  );
}
```

Full flow summary

```
Client books → POST /api/bookings
  ├── prisma.session.create()
  ├── Resend → confirmation email
  ├── Google Calendar event created (via access token)
  └── Admin notified

Vercel Cron (every 15 min) → GET /api/cron/session-reminders
  └── finds sessions starting in ~60 min → Resend reminder → marks reminderSentAt

Client visits /portal → sees upcoming / past / cancelled
  └── Cancel → DELETE /api/bookings/[id]
        ├── prisma update status='cancelled'
        └── Google Calendar event deleted
```

Everything plugs into your existing Prisma, NextAuth, and Resend stack with no new services. Want me to build the `reminderEmailHTML` as a full React Email component next, or tackle the `/portal` UI styling to match VizTR's brand system?

can you elaborate the features you developed in above conversation in bullet points

Catalogued seven interconnected streaming infrastructure components built throughout conversation

Here's everything built in this conversation:

Pixel Streaming infrastructure

- Full architecture diagram — UE5 → Cirrus → Cloudflare Tunnel → browser WebRTC flow
- UE5 Pixel Streaming plugin setup with all launch flags (`-PixelStreamingIP`, `-RenderOffScreen`, `-ResX/Y`, etc.)
- Cirrus signalling server install and run commands from the UE5 engine directory
- PM2 process manager config to keep Cirrus alive and auto-start on PC boot
- Cloudflare Tunnel setup exposing `wss://stream.viztr.io` without static IP or port forwarding
- coturn TURN/STUN server config on a VPS for NAT traversal
- Startup order checklist for demo sessions

Next.js Pixel Streaming component

- `PixelStreamingViewer.tsx` client component using `@epicgames/epicpixelstreaming`
- Auto-connect, auto-play, hover mouse mode flags
- Live / connecting / error status badge with animated dot
- Retry connection button on stream failure
- `dynamic()` import with `ssr: false` for the `/xr` page
- `NEXT_PUBLIC_PS_SIGNALING_URL` environment variable wiring

Stream admin panel

- Interactive live dashboard widget (simulates real session behaviour — joins, disconnects, TURN fallback warnings firing in real time)
- Metric cards — stream status, active viewers, bitrate, GPU usage + VRAM
- UE5 / Cirrus / Cloudflare Tunnel / TURN health rows
- Kill stream and restart buttons
- Bandwidth bar chart (outbound + WebRTC overhead)
- Active sessions list with live timers
- Event log with timestamped entries
- `GET /api/admin/pixel-streaming/status` route — polls metrics sidecar, admin-only via NextAuth role check
- `POST /api/admin/pixel-streaming/control` route — kill / restart commands
- Node.js metrics sidecar (`ps-metrics-server.js`) running on the GPU workstation — parses `nvidia-smi` for GPU/VRAM, exposes health over HTTP, executes PM2 commands on control requests
- `PS_METRICS_URL` + `PS_METRICS_SECRET` env vars for secure Vercel → workstation tunnel

Session booking system

- Full interactive booking widget — four-step flow: service picker → calendar + time slot → client details → confirmation
- Four service types with durations: VR walkthrough, exterior visualisation, interior XR tour, master plan review
- Calendar with weekend / past-date locking, green availability dots, month navigation
- Time slot grid with pre-marked booked slots (conflict-aware)
- Client detail form — name, email, company, project type, notes
- Confirmation screen with generated session ID (`VZT-XXXXXX`)
- `POST /api/bookings` — clash check, `prisma.session.create()`, Resend confirmation email to client, admin alert email
- `GET /api/bookings?date=` — returns booked slots for a given date (used to grey out the calendar)
- Prisma `Session` model with all booking fields
- Public `/book` page with metadata

Google Calendar sync

- `lib/google-calendar.ts` — creates a Google Calendar event using the client's OAuth access token from NextAuth
- Event includes session details, project type, join stream link, and 60-minute popup + email reminders
- `calendar.events` scope added to NextAuth Google provider with `offline` access for refresh tokens
- `gcalEventId` saved to Prisma so events can be updated or deleted later
- Cancel flow deletes the Google Calendar event via `DELETE /api/bookings/[id]`

Automated reminder emails — Vercel Cron

- `vercel.json` cron schedule every 15 minutes
- `GET /api/cron/session-reminders` — queries sessions starting in the next 60–75 minute window with no reminder sent yet
- Sends Resend reminder email per session, then stamps `reminderSentAt` to prevent duplicates
- `CRON_SECRET` header auth to block external calls
- HTML reminder email template with date, time, session ID, and portal link
- `startAt` DateTime field and `reminderSentAt` nullable field added to Prisma schema

Client portal

- Full interactive portal widget — profile bar, stats (total sessions, upcoming count, hours streamed), reminder banner, tabbed session list (upcoming / past / cancelled)
- Session cards with service name, project, date/time/duration grid, session ID, status badge
- "Live now" section separator for active sessions with join stream button

- Add to Google Calendar / calendar added state per session
- Cancel flow with confirmation overlay — moves session to Cancelled tab on confirm, updates tab counts live
- Rebook and receipt download buttons on past/cancelled sessions
- `app/portal/page.tsx` — server component, NextAuth-gated, fetches all sessions for logged-in email, splits into upcoming / past / cancelled
- `SessionCard.tsx` client component with inline cancel confirm UI, calls `DELETE /api/bookings/[id]`, then `router.refresh()` to re-fetch server data

now give me the .bat file which connects the local gpu to the pixel streaming online server of our website.

Architected comprehensive batch script orchestrating GPU, server, and streaming components

Three `.bat` files — here's exactly what each does and how to use them.

Files

`viztr-stream-setup.bat` — run this once on a fresh machine. It installs PM2 globally, installs `cloudflared` via winget, logs you into Cloudflare, creates the `viztr-pixel` tunnel, and routes `stream.viztr.io` and `metrics.viztr.io` DNS to it.

`viztr-stream-start.bat` — your daily launcher. Double-click before any client demo. It runs 6 steps in sequence:

1. Checks Node.js, PM2, cloudflared, your UE5 exe, and Cirrus all exist — hard stops with a clear error if anything is missing
2. Kills any stale leftover processes from a previous session
3. Starts Cirrus signalling server via PM2 on ports 80 (HTTP) and 8888 (streamer) — installs `node_modules` automatically on first run
4. Starts the metrics sidecar via PM2 on port 9000
5. Opens Cloudflare Tunnel in a minimised window pointing to `localhost:80`
6. Launches your packaged UE5 build with all Pixel Streaming flags, waits 12 seconds for it to register with Cirrus, then prints a live status summary

`viztr-stream-stop.bat` — clean shutdown. Deletes both PM2 processes, kills `cloudflared.exe`, and kills the UE5 process. Run this after every session.

Before first use — edit the CONFIG block in `viztr-stream-start.bat`

bat

```
set UE5_EXE=C:\VizTR\Build\Windows\VizTR.exe        ← your actual packaged build path
set CIRRUS_DIR=C:\Program Files\Epic Games\UE_5.4\...  ← match your UE version
set CF_TUNNEL_NAME=viztr-pixel                      ← must match what setup created
set METRICS_DIR=C:\VizTR\ps-metrics-server          ← where you put ps-metrics-serv
set METRICS_SECRET=your-long-random-secret-here     ← must match Vercel env var
```

Change `UE_5.4` in `CIRRUS_DIR` to whichever UE version you have installed (`UE_5.3`, `UE_5.5`, etc.).

Daily workflow

```
viztr-stream-setup.bat   ← once only on new machine

viztr-stream-start.bat   ← before every demo session
   ... client streams ...
viztr-stream-stop.bat    ← after session ends
```

viztr-stream-start.bat Download

viztr-stream-stop.bat Download

viztr-stream-setup.bat Download

Download all

build the html artifact of pixel streaming as discussed above

Architected luxury dark interface with WebSocket streaming integration

continue

Synthesized technical documentation for streaming viewer implementation

Here's what's inside the file and how to drop it into VizTR.

What the file contains

Header bar — VizTR logo, session name + type, live viewer count badge, and a status pill that cycles between Connecting (gold) / Live (green) / Offline (red) with an animated dot.

Stream stage — full-viewport video area with three overlay states:

- **Connecting —** animated blueprint grid with a horizontal scan line, dual expanding rings around the VizTR logo mark, and per-character dot animation. This is your signature architectural drawing aesthetic while the WebRTC handshake happens.
- **Live —** the UE5 video stream fills the stage. Subtle gradient vignettes top and bottom. A LIVE badge appears in the corner on hover.
- **Error —** clean error overlay with the specific failure reason (WS refused, WebRTC dropped, etc.) and a retry button that cleanly tears down the old connection and reconnects.

Controls bar — audio toggle (with muted icon swap), quality picker dropdown (Ultra / High / Standard / Low with bitrate labels), session ID chip with one-click copy, stats toggle, settings toggle, fullscreen toggle. Keyboard shortcut: `F11` for fullscreen, `M` to mute.

Stats panel — slides up above the controls bar. Shows bitrate, FPS, latency, resolution, connection type, and codec — all pulled from real `RTCPeerConnection.getStats()`, colour-coded green / amber / red by threshold.

Settings panel — slides in from the right. Quality, auto-quality toggle, target bitrate, audio on/off, mouse capture (pointer lock), keyboard pass-through, touch controls, and a full connection info section (session ID, signalling URL, region, connection type badge P2P vs Relay).

Sim canvas — while there's no real stream (demo mode or offline), an animated architectural floor plan renders — blueprint grid, golden-section room outlines, door arcs, dimension markers, north arrow compass. Stops and hides the moment a WebRTC video track arrives.

How to use it in VizTR

Drop it into Next.js as a static page:

```bash
# Put it in /public so it's served as-is
mv viztr-pixel-streaming.html public/xr-viewer.html
# Access at: https://viztr.io/xr-viewer.html
```

Or embed it in an iframe inside your `/xr` Next.js page:

```tsx
```

```
// app/xr/page.tsx
export default function XRPage() {
  return (
    <iframe
      src="/xr-viewer.html"
      className="w-full h-screen border-0"
      allow="autoplay; fullscreen; camera; microphone"
      allowFullScreen
    />
  );
}
```

Before going live — two values to change at the top of the `<script>` block:

```
const CFG = {
  signalingUrl : 'wss://stream.viztr.io',       // ← already correct
  sessionId    : 'VZT-A4F2B1',                  // ← inject dynamically per booking
  sessionName  : 'Skyline Villa',                // ← inject from booking record
  sessionType  : 'VR Walkthrough',               // ← inject from booking record
  iceServers   : [
    { urls: 'stun:stun.l.google.com:19302' },
    {
      urls       : 'turn:turn.viztr.io:3478',
      username   : 'viztr',
      credential : 'REPLACE_WITH_COTURN_PASSWORD'  // ← your coturn password
    }
  ]
};
```

For dynamic injection from the booking system, pass them as query params:

```
/xr-viewer.html?session=VZT-A4F2B1&name=Skyline+Villa&type=VR+Walkthrough
```

Then read them in JS at the top of the script:

```
const p = new URLSearchParams(location.search);
CFG.sessionId   = p.get('session') || 'VZT-DEMO';
CFG.sessionName = p.get('name')    || 'Live Demo';
CFG.sessionType = p.get('type')    || 'VR Walkthrough';
```

Want me to add the query param injection + a session-expired overlay (auto-fires after the booked duration ends) next?

Viztr pixel streaming

Code · HTML

Download